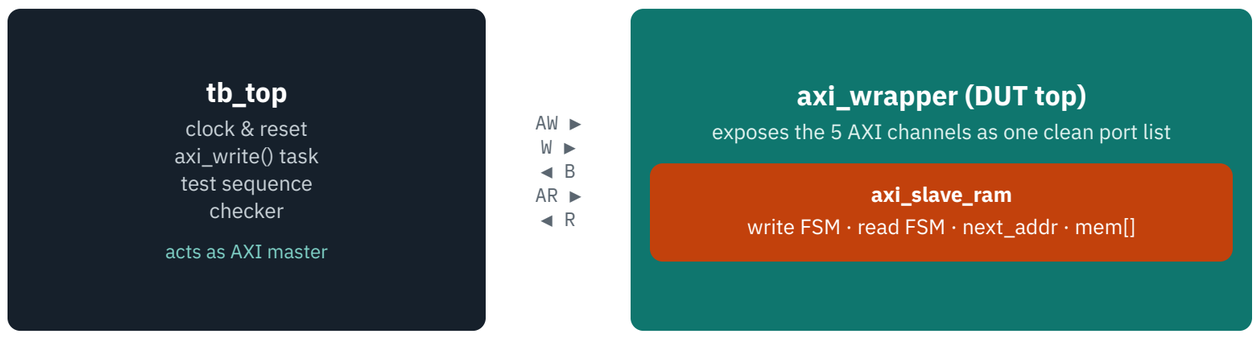
DV INTERVIEW · GLANCE SHEET 9 · PROJECT PART 2

Writing to the RAM from a Testbench

The testbench plays AXI master: it drives AW and W, waits for the handshakes, collects B, then checks memory.

```
drive with <= right after posedge
sample READY at the next posedge
AW ‖ W in fork…join, then B
```

1 Testbench structure & the AXI wrapper



- **Wrapper** = a thin top module that groups the AXI ports and instantiates the RAM slave. The TB only ever talks to the wrapper, so the core can change without touching the TB.
- Same idea in UVM: the wrapper's ports bind to an **AXI interface**, and drivers/monitors use that interface.

2 Clock, reset & idle values

```
initial ACLK = 0;
always #5 ACLK = ~ACLK;          // 100 MHz

initial begin
  ARESETn = 0;
  AWVALID = 0; WVALID = 0; WLAST = 0;
  BREADY  = 0; ARVALID = 0; RREADY = 0;
  repeat (5) @(posedge ACLK);
  ARESETn <= 1;                  // de-assert on an edge
  @(posedge ACLK);
  run_write_tests();
  $finish;
end
```

All master VALIDs must be LOW during reset and stay LOW until after ARESETn goes HIGH — the TB is the master, so this is **your** responsibility.

3 The reusable `axi_write` task (SystemVerilog)

```
logic [DATA_W-1:0] wdata_q[$];   // filled by the test before calling
logic [STRB_W-1:0] wstrb_q[$];

task automatic axi_write(input [ID_W-1:0] id, input [ADDR_W-1:0] addr,
                         input [7:0] len, input [2:0] size, input [1:0] burst);
  fork
    begin : aw_ch                                  // ---- address channel
      AWID <= id;  AWADDR <= addr;  AWLEN <= len;
      AWSIZE <= size;  AWBURST <= burst;  AWVALID <= 1;
      do @(posedge ACLK); while (!AWREADY);      // handshake edge
      AWVALID <= 0;
    end
    begin : w_ch                                   // ---- data channel
      for (int i = 0; i <= len; i++) begin
        WDATA <= wdata_q[i];  WSTRB <= wstrb_q[i];
        WLAST <= (i == len);  WVALID <= 1;
        do @(posedge ACLK); while (!WREADY);     // one beat accepted
      end
      WVALID <= 0;  WLAST <= 0;
    end
  join

  BREADY <= 1;                                     // ---- response channel
  do @(posedge ACLK); while (!BVALID);
  BREADY <= 0;
  if (BID !== id)      $error("BID %0h != AWID %0h", BID, id);
  if (BRESP !== 2'b00) $error("BRESP = %b at addr %h", BRESP, addr);
endtask
```

fork…join runs AW and W at the same time. The TB raises WVALID without waiting for AWREADY — exactly what the protocol requires of a master (no "wait for READY before VALID").

do @(posedge) while(!READY) — after the edge, READY still shows the value the DUT had *at* that edge (the DUT's registers update later via NBA), so this samples the handshake correctly.

Non-blocking `<=` for every drive avoids races with the DUT on the same clock edge.

Payload is held automatically: nothing changes until the loop sees READY, so VALID-stable / data-stable rules hold.

Add a **timeout** around each wait (e.g. a parallel `repeat(1000) @(posedge ACLK)` + `$fatal`) so a stuck READY doesn't hang the sim forever.

4 What a 2-beat write looks like (with the Part-1 slave)

ACLK	T1	T2	T3	T4	T5	T6	T7	T8
AWVALID	⇄	0	0	0	0	0	0	0
AWREADY	⇄	0	0	0	1	1	1	1
WVALID	1	⇄	⇄	0	0	0	0	0
WREADY	0	⇄	⇄	0	0	0	0	0
WDATA	D0	D0	D1	–	–	–	–	–
WLAST	0	0	1	0	0	0	0	0
BVALID	0	0	0	⇄	0	0	0	0
BREADY	0	0	0	⇄	0	0	0	0

- **T1** AW accepted (slave idle). W is already offered but WREADY is LOW — slave waits for AW first.
- **T2, T3** two beats; WLAST with D1.
- **T4** B handshake → slave back to IDLE, AWREADY HIGH again at T5.

5 Write tests & expected memory (32-bit RAM)

Test	Call	Expect
Single	`addr 0x00, len 0, size 2, INCR`, D = 0xCAFE_F00D	mem[0] = CAFEF00D
INCR ×4	`addr 0x10, len 3`, D = 1,2,3,4	mem[4..7] = 1,2,3,4
WRAP ×4	`addr 0x38, len 3, WRAP`, D = A,B,C,D	mem[14]=A, [15]=B, [12]=C, [13]=D
FIXED ×3	`addr 0x40, len 2, FIXED`, D = X,Y,Z	mem[16] = Z (last wins)
Unaligned	`addr 0x51, len 1`, strb 1110, 1111	mem[20] bytes 1–3, mem[21] all
BREADY delay	hold BREADY LOW 5 cycles	BVALID stays HIGH, no lost response

PARTIAL-STROBE EXAMPLE

mem before	11	22	33	44	
WDATA	AA	BB	CC	DD	WSTRB = 0101
mem after	11	BB	33	DD	= 0x11BB33DD

6 Checking the write

OPTION A · PEEK MEMORY (QUICK)

```
if (tb_top.dut.u_ram.mem[addr >> 2] !== exp)
  $error("mem[%h] = %h, exp %h",
         addr, tb_top.dut.u_ram.mem[addr >> 2], exp);
```

OPTION B · READ BACK OVER AXI (BETTER)

Write, then `axi_read()` the same address and compare — tests the read path too and doesn't depend on internal names.

OPTION C · REFERENCE MODEL

Keep a byte-array model in the TB; apply every write with WSTRB; compare on every read. This is the scoreboard idea, scaled up in UVM.

Always check **BRESP** and **BID** too — a "passing" write with SLVERR is a bug.

! Common testbench mistakes

- Waiting for AWREADY/WREADY **before** raising VALID → can deadlock with some slaves.
- Using blocking `=` on DUT inputs at posedge → race conditions.
- Checking READY with `wait(AWREADY)` (level) instead of at a clock edge → miscounted beats.
- Forgetting WLAST, or setting it on every beat.
- Driving AW and W strictly in sequence only — never exercises W-before-AW.
- Leaving BREADY LOW forever → write never completes.
- Not filling `wdata_q` with LEN+1 entries.
- No timeout → hung simulation instead of a clear failure.
- Checking memory before B handshake has completed.

? Interview Q&A

Question	Answer
Why fork AW and W?	Channels are independent; a master mustn't serialize them.
Why NBA in the TB?	Avoids races with DUT flops at the same edge.
How do you know a beat was taken?	VALID && READY sampled at posedge.
What's an AXI wrapper?	Top that exposes AXI ports around the core.
How to check the write?	Read back / reference model + BRESP.
Next step to UVM?	Task → driver, checks → monitor + scoreboard, tests → sequences.